

Lab 9:

Htax_tx_monitor:

```
class htax_tx_monitor_c extends uvm_monitor;

    parameter PORTS = `PORTS;

    `uvm_component_utils(htax_tx_monitor_c)

    uvm_analysis_port #(htax_tx_mon_packet_c) tx_collect_port;

    virtual interface htax_tx_interface htax_tx_intf;

    htax_tx_mon_packet_c tx_mon_packet;

    int pkt_len;

    covergroup cover_htax_packet;
        option.per_instance = 1;
        option.name = "cover_htax_packet";

        // Coverpoint for htax packet field : destination port
        DEST_PORT : coverpoint tx_mon_packet.dest_port {bins dest_port[] = {[0:3]};}

        // TO DO : Coverpoint for htax packet field : vc (include vc=0 in illegal bin)
        VC: coverpoint tx_mon_packet.vc {illegal_bins b1= {0}; bins vc = {[0:$]};}

        // TO DO : Coverpoint for htax packet field : length (Divide range [3:63] into 16 bins)
        LENGTH: coverpoint tx_mon_packet.length { bins b7[16]= {[3:63]};}
```

```
// Coverpoints for Crosses
```

```
// TO DO : DEST_PORT cross VC
```

```
cross_DEST_PORT_VC : cross DEST_PORT, VC;
```

```
// TO DO : DEST_PORT cross LENGTH
```

```
cross_DEST_PORT_LENGTH : cross DEST_PORT, LENGTH;
```

```
// TO DO : VC cross LENGTH
```

```
cross_VC_LENGTH : cross VC, LENGTH;
```

```
endgroup
```

```
covergroup cover_htax_tx_intf;
```

```
option.per_instance = 1;
```

```
option.name = "cover_htax_tx_intf";
```

```
// TO DO : Coverpoint for tx_outport_req: covered all the values
```

```
0001,0010,0100,1000
```

```
c1: coverpoint htax_tx_intf.tx_outport_req{bins b1 = {1,2,4,8}; ignore_bins b2 =  
{0};}
```

```
// TO DO : Coverpoint for tx_vc_req: All the VCs are requested atleast once.
```

```
Ignore what is not allowed, or put it as illegal
```

```
c2: coverpoint htax_tx_intf.tx_vc_req {bins b3 = {1,2,3}; illegal_bins b4 = {0};}
```

```
// TO DO : Coverpoint for tx_vc_gnt: All the virtual channels are granted atleast
```

```
once.
```

```
c3: coverpoint htax_tx_intf.tx_vc_gnt {bins b5 = {1,2,3}; illegal_bins b6 = {0};}
```

```
endgroup
```

```
//constructor
```

```
function new (string name, uvm_component parent);
```

```
    super.new(name,parent);
```

```
    tx_collect_port = new("tx_collect_port",this);
```

```
    tx_mon_packet      = new();
```

```
    //Instance for the covergroup cover_htax_packet
```

```
    this.cover_htax_packet = new();
```

```
    //Instance for the covergroup cover_htax_tx_intf
```

```
    this.cover_htax_tx_intf = new();
```

```
endfunction : new
```

```
//UVM build phase
```

```
function void build_phase(uvm_phase phase);
```

```
    super.build_phase(phase);
```

```
    if(!uvm_config_db#(virtual htax_tx_interface)::get(this,"","tx_vif",htax_tx_intf))
```

```
        `uvm_fatal("NO_TX_VIF",{ "Virtual Interface needs to be set for ",  
get_full_name(),".tx_vif" })
```

```
endfunction : build_phase
```

```
task run_phase(uvm_phase phase);
```

```
    forever begin
```

```
        pkt_len=0;
```

```
        //Assign tx_mon_packet.dest_port from htax_tx_intf.tx_outport_req
```

```
        @(posedge |htax_tx_intf.tx_vc_gnt) begin
```

```

        for(int i=0; i < PORTS; i++)
            if(htax_tx_intf.tx_outport_req[i]==1'b1)
                tx_mon_packet.dest_port = i;

        //Assign tx_vc_req to tx_mon_packet.vc
        tx_mon_packet.vc = htax_tx_intf.tx_vc_req;

        cover_htax_tx_intf.sample();    //Sample Coverage on
htax_tx_intf

    end

    @(posedge htax_tx_intf.clk)

    //On consecutive cycles append htax_tx_intf.tx_data to
tx_mon_packet.data[] till htax_tx_intf.tx_eot pulse
    while(htax_tx_intf.tx_eot==0) begin
        @(posedge htax_tx_intf.clk)
        tx_mon_packet.data = new[++pkt_len]
(tx_mon_packet.data);

        tx_mon_packet.data[pkt_len-1]=htax_tx_intf.tx_data;

    end

    //Assign pkt_len to tx_mon_packet.length
    tx_mon_packet.length = pkt_len;
    tx_collect_port.write(tx_mon_packet);
    cover_htax_packet.sample();    //Sample Coverage on tx_mon_packet

    end

    endtask : run_phase

endclass : htax_tx_monitor_c

```

Htax_rx_monitor:

```
class htax_rx_monitor_c extends uvm_monitor;

    `uvm_component_utils(htax_rx_monitor_c)

    //Analysis port to communicate with Scoreboard
    uvm_analysis_port #(htax_rx_mon_packet_c) rx_collect_port;

    virtual interface htax_rx_interface htax_rx_intf;
    htax_rx_mon_packet_c rx_mon_packet;
    int pkt_len;

    // TO DO : Create covergroup for htax_rx_intf and add at least one coverpoint
    covergroup cover_htax_rx_intf;
        option.per_instance = 1;
        option.name = "cover_htax_rx_intf";

        // Coverpoint for rx_vc_gnt: All the virtual channels are granted atleast once.
        c1: coverpoint htax_rx_intf.rx_vc_gnt {bins b1 = {1,2,3}; illegal_bins b2 = {0};}

        // Coverpoint for rx_vc_req: All the VCs are requested atleast once. Ignore what is not
        // allowed, or put it as illegal
        c2: coverpoint htax_rx_intf.rx_vc_req {bins b3 = {1,2,3}; illegal_bins b4 = {0};}
    endgroup

    function new (string name, uvm_component parent);
        super.new(name, parent);
        rx_collect_port = new ("rx_collect_port", this);
        rx_mon_packet    = new();
    endfunction
endclass
```

```

endfunction : new

function void build_phase (uvm_phase phase);
    super.build_phase(phase);
    if(!uvm_config_db#(virtual htax_rx_interface)::get(this,"","rx_vif",htax_rx_intf))
        `uvm_fatal("RX_VIF",{ "Virtual Interface needs to be set for
",get_full_name(),".rx_vif"})
    endfunction : build_phase

task run_phase(uvm_phase phase);
    forever begin
        pkt_len=0;

        //Wait for rising edge of htax_rx_intf.rx_sot
        @(posedge (|htax_rx_intf.rx_sot))

        //On consecutive cycles append htax_rx_intf.rx_data to
rx_mon_packet.data[] till htax_rx_intf.rx_eot pulse
        while(!htax_rx_intf.rx_eot==1) begin
            @(posedge htax_rx_intf.clk);
            rx_mon_packet.data = new [++pkt_len] (rx_mon_packet.data);
            rx_mon_packet.data[pkt_len-1] = htax_rx_intf.rx_data;
        end

        //Write the rx_mon_packet on anaylysis port
        rx_collect_port.write(rx_mon_packet);

    end

endtask : run_phase
endclass : htax_rx_monitor_c

```

UVM Report:

```
--- UVM Report catcher Summary ---

Number of demoted UVM_FATAL reports : 0
Number of demoted UVM_ERROR reports : 0
Number of demoted UVM_WARNING reports: 0
Number of caught UVM_FATAL reports : 0
Number of caught UVM_ERROR reports : 0
Number of caught UVM_WARNING reports : 0

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 107
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[RNIST] 1
[SCOREBOARD] 65
[TEST_DONE] 1
[TOP] 5
[UVMTOP] 1
[htax_tx_driver_c] 30
[simple_random_seq] 3
[simple_random_test] 1
Simulation complete via $finish(1) at time 63030 NS + 45
/opt/coe/cadence/XCELIUM/tools/methodology/UVM/CDNS-1.ld/sv/src/base/uvm_root.svh:457 $finish;
xcelium> exit
```

Coverage Report:

top	65.98%	425 / 678 (62.68%)	25%
inst_htax_tx_intf[3]	1.28%	2 / 85 (2.35%)	0%
inst_htax_tx_intf[2]	1.28%	2 / 85 (2.35%)	0%
inst_htax_tx_intf[1]	100%	85 / 85 (100%)	100%
assert_tx_rel_gnt_tx_eot	100%	1 / 1 (100%)	100%
assert_tx_eot_single_cycle	100%	1 / 1 (100%)	100%
assert_tx_vc_sot_vc_gnt_1	100%	1 / 1 (100%)	100%
assert_tx_vc_sot_vc_gnt_0	100%	1 / 1 (100%)	100%
assert_tx_vc_req_outport_req	100%	1 / 1 (100%)	100%
assert_tx_outport_req_vc_req	100%	1 / 1 (100%)	100%
assert_tx_outport_req_one_hot	100%	1 / 1 (100%)	100%
inst_htax_tx_intf[0]	1.28%	2 / 85 (2.35%)	0%
inst_htax_rx_intf[3]		79 / 79 (100%)	n/a
inst_htax_rx_intf[2]	100%	79 / 79 (100%)	n/a
inst_htax_rx_intf[1]	100%	79 / 79 (100%)	n/a
inst_htax_rx_intf[0]	100%	79 / 79 (100%)	n/a